

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: CPU 设计之指令拓展

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 张立冬、马致远

实验地点: 东 4-509 实验日期: 2024 年 11 月 20 日

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心: 指令执行过程与控制流关系
3. 拓展设计 Datapath 和 Control Unit
4. 设计测试程序

二、实验内容和原理

内容:

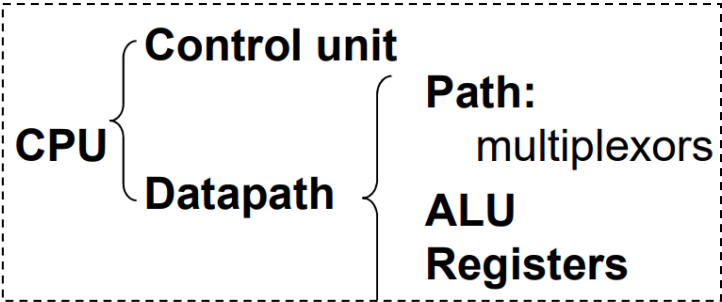
任务一: 重新设计数据通路和控制器

- 兼容 Lab4-1/2 中的数据通路与控制器
- 拓展指令

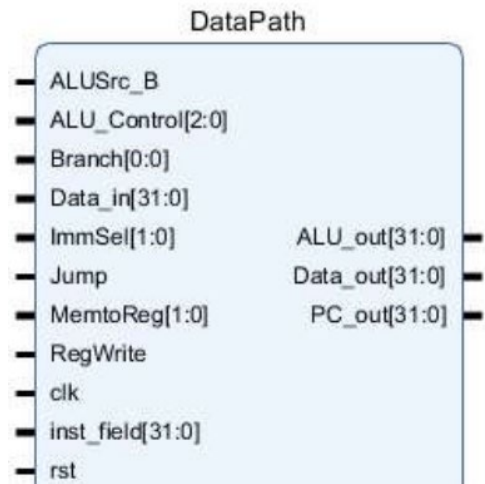
任务二: 设计指令集测试方案并完成测试

原理：

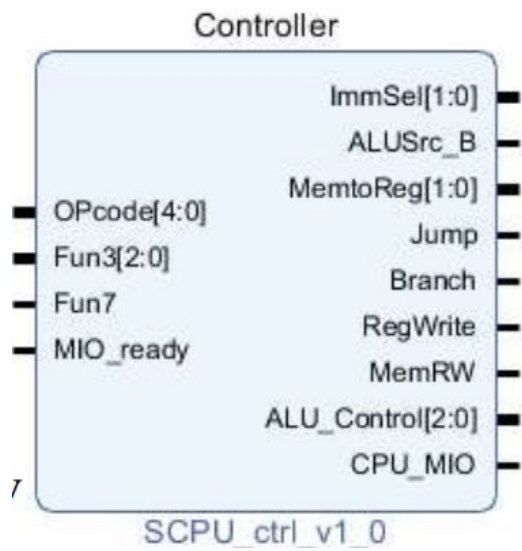
2.1 计算机系统中 CPU 组成



• 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



• 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。



2.2 RISC-V 指令拓展

前两实验中的 Datapath 和 SCPU_ctrl 实现了 R、I、S、B、J 型指令中的若干指令（黑）：

R : add , sub , and , or , xor , srl , slt ; **sltu , sra , sll**

I : addi , andi , ori , xori , srli , slti , lw ; **sltiu , srai , slli , jalr**

S : sw

B : beq , **bne**

J : jal

Lab4-03 中需要拓展以上 5 类指令，并添加 U-Type 中的 **lui**。

jalr rd, (immediate) rs1 无条件跳转-链接（寄存器地址）

功能: $rd = pc+4$, $pc \leftarrow rs1 + \text{immediate}$

(1) jalr (I-type):

$Rd = PC + 4$ ，写回 Registers file 的数据需添加 PC+4，即为控制 Write data 的 MUX 控制信号 MemtoReg 添加位宽（后面的 lui 也要增加）。

$PC = rs1 + \text{immediate}$ 由 ALU_res 得出，现在下一条 PC 的可能取值有 3 种，因此需要添加 Jump 信号的位宽。

lui rd, immediate

功能: $rd = \text{immediate} \ll 12$

Eg: lui x5,0x12345

$X5 = 0x12345000$ 取左移12位的20位立即数

(2) lui (U-type):

大立即数生成意味着 ImmGen 的控制信号 ImmSel 需要添加新的译码类型。

$Rd = \text{immediate}$ 由 ImmGen 得出，即为控制 Write data 的 MUX 控制信号 MemtoReg 添加位宽。

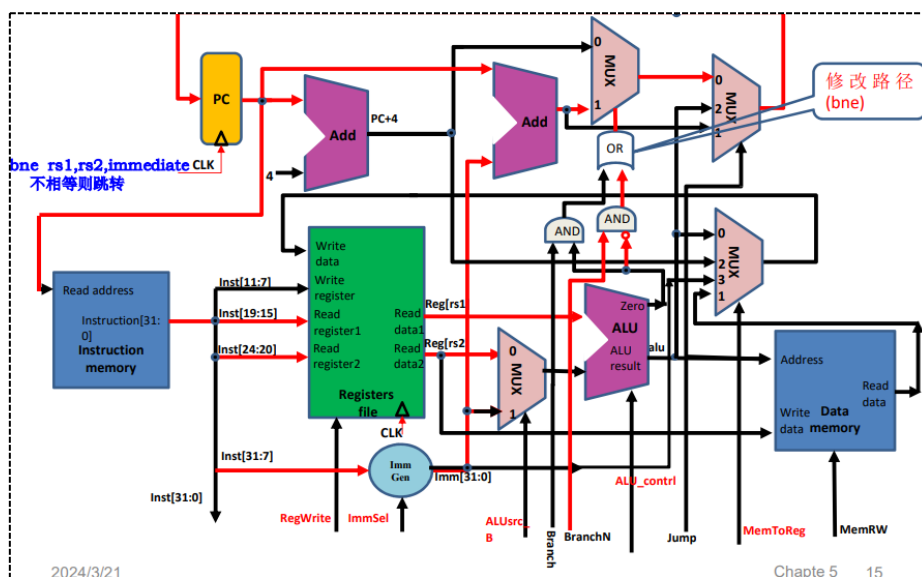
(3) bne (B-Type):

【Bne rs1 , rs2 , immediate （当 rs1 != rs2 时 ， 跳转到 PC + immediate）】

首先需要添加一个类似 Branch 的控制信号 BranchN。

Beq 判定跳转的条件是 zero & Branch = 1 （利用一个 and），如果不添加控制信号，Branch = 1 、 zero = 0 无法实现 bne 跳转。

因此添加 BranchN，在判定 PC 跳转的 MUX 处，添加一个 or，使得 BranchN = 1、zero = 0 时也能够利用 BranchN & (! zero) 来判定 bne 的跳转。



2.3 指令拓展后的数据通路和控制器

2.3.1 New 数据通路:

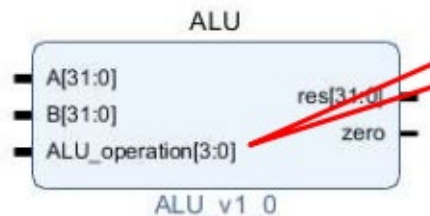
主要的修改端口在 Lab4-01 的思考题与 2.2 中提及。

(1) New 【 ALU 】 :

新添加操作 slt、sra、sll，ALU_operation 位宽变为 4。

```
`timescale 1ns / 1ps
```

```
module ALU(
    input [31:0] A,
    input [3:0] ALU_operation,
    input [31:0] B,
    output reg [31:0] res,
    output zero
);
    parameter one = 32'h00000001, zero_0 = 32'h00000000;
    always @ (*)
```



```

    case (ALU_operation)
    4'b0000: res = A & B; //and
    4'b0001: res = A | B; //or
    4'b0010: res = A + B; //add
    4'b1100: res = A ^ B; //xor
    4'b1101: res = A >> B; //sr1
    4'b0110: res = A - B; //sub
    4'b0111: res = ($signed(A) < $signed(B)) ? one : zero_0; //slt
    4'b1111: res = ($signed(A) >> B); //sra
    4'b1110: res = A << B; //sll
    4'b1001: res = (A < B) ? one : zero_0; //sltu
    default: res = 32'hx;
    endcase
    assign zero = (res==0)? 1: 0;
endmodule

```

(2) New 【 ImmGen 】:

新添加 U-Type, 生成一个 20 位立即数, 同时 ImmSel 的位宽需要改为 3。

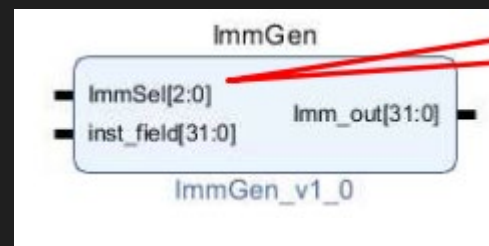
```
`timescale 1ns / 1ps
```

```

module ImmGen_more(
    input wire [2:0] ImmSel,
    input wire [31:0] inst_field,
    output reg [31:0] Imm_out
);

always@*begin
    case(ImmSel)
    3'b000:Imm_out = {inst_field[31:12],12'h000}; // U-type
    3'b001:Imm_out = {{20{inst_field[31]}},inst_field[31:20]}; //addi\lw(I)
    3'b010:Imm_out =
    {{20{inst_field[31]}},inst_field[31:25],inst_field[11:7]}; //sw(s)
    3'b011:Imm_out =
    {{20{inst_field[31]}},inst_field[7],inst_field[30:25],inst_field[11:8],
    1'b0}; //beq(b)
    3'b100:Imm_out =
    {{12{inst_field[31]}},inst_field[19:12],inst_field[20],inst_field[30:21
    ],1'b0}; //jal(j)
    endcase
    end
endmodule

```



2.3.2 New 控制器:

信号	源数目	功能定义	赋值0时动作	赋值1时动作	赋值2时动作	赋值3时动作
ALUSrc_B	2	ALU端口B输入选择	选择源操作数寄存器2数据	选择32位立即数(符号扩展后)	-	-
MemToReg	4	寄存器写入数据选择	选择ALU输出	选择存储器数据	选择PC+4	选择imm
Branch	2	Beq指令目标地址选择	选择PC+4地址	选择转移目的地址PC+imm (zero=1)	-	-
BranchN	2	Bne指令目标地址选择	选择PC+4地址	选择转移目的地址PC+imm (zero=0)	-	-
Jump	3	J指令目标地址选择	由Branch决定输出	选择跳转目标地址PC+imm (JAL)	选择跳转目标地址ALU输出(JALR:rs1+imm)	-
RegWrite	-	寄存器写控制	禁止寄存器写	使能寄存器写	-	-
MemRW	-	存储器读写控制	存储器读使能, 存储器写禁止	存储器写使能, 存储器读禁止	-	-
ALU_Control	0000 - 1111	4位ALU操作控制	参考表ALU_Control			
ImmSel	000-111	3位立即数组合控制	参考表ImmSel			

【控制信号定义】

Inst[31:0]	Banch	Branch N	Jump	ImmSel	ALUSrc_B	ALU_Control	MemRW	RegWrite	MemtoReg
add	0	0	0	*	Reg (0)	Add	Read	1	ALU(0)
sub	0	0	0	*	Reg (0)	Sub	Read	1	ALU(0)
(R-R Op)	0	0	0	*	Reg (0)	(Op)	Read	1	ALU(0)
addi	0	0	0	I	Imm (1)	Add	Read	1	ALU(0)
lw	0	0	0	I	Imm (1)	Add	Read	1	Mem(1)
sw	0	0	0	S	Imm (1)	Add	Write	0	*
beq	0	0	0	B	Reg (0)	sub	*	0	*
beq	1	*	0	B	Reg (0)	sub	*	0	*
bne	0	0	0	B	Reg (0)	sub	*	0	*
bne	*	1	0	B	Reg (0)	sub	*	0	*
jalr	*	*	2	I	Imm (1)	Add	*	1	PC+4(2)
jal	*	*	1	J	Imm (1)	*	*	1	PC+4(2)
lui	0	0	0	U	*	*	*	1	Imm(3)

【控制信号真值表】

Instruction type	Instruction opcode[6:0]	Instruction operation	(sign-extend)immediate	Imm Sel
I-type	0000011	Lw;bu;lh; lb;lhu	(sign-extend) instr[31:20]	001
	0010011	Addi;slli;slli u;xori;ori;a ndi;slli;srai		
	1100111	jalr		
S-type	0100011	Sw;sb;sh	(sign-extend) instr[31:25],[11:7]	010
B-type	1100011	Beq;bne;blt bge;bltu;b geu	(sign-extend) instr[31],[7],[30:25],[11:8], 1'b0	011
J-type	1101111	jal	(sign-extend) instr[31],[19:12],[20],[30:21],1 'b0	100
U-type	0010111	jalr	instr[31:12],12'h000	000
	0110111	lui		

【ImmSel 控制信号】

三、实验过程和数据记录

3.1 设计实现 Datapath:

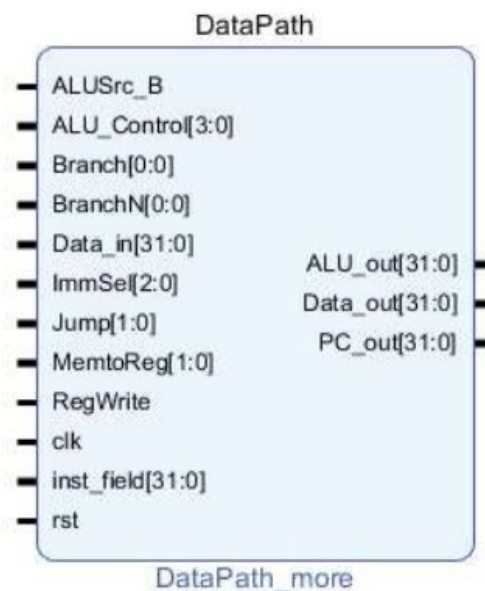
(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-Datapath_more;

本实验仍然需要用到 Lab0 设计的 MUX 多路选择器、若干基本运算模块 (and、or、xor、nor、srl 等)，以及 Lab1 设计的 Regs。

同时需要修改 ALU、ImmGen_more 两个模块 (具体代码见实验原理)。

(2) 设计 CPU 中的 Datapath:



代码实现如下:

```
`timescale 1ns / 1ps
module Datapath_more(
    input wire clk,
    input wire rst,
    input wire[31:0] inst_field,
    input wire[31:0] Data_in,
    input wire[3:0] ALU_operation,
    input wire[2:0] ImmSel,
    input wire[1:0] MemtoReg,
    input wire ALUSrc_B,
    input wire [1:0] Jump,
    input wire Branch, // beq
    input wire BranchN, // bne
    input wire RegWrite,
```



```

output wire[31:0] PC_out,
output wire[31:0] Data_out,
output wire[31:0] ALU_out,
output wire[1023:0] all_regs_data
);

wire[31:0] Imm_out;
wire[31:0] PC_inc;
wire[31:0] PC_imm;
wire ALU_zero;
wire[31:0] PC_1;
wire[31:0] Wt_data;
wire[31:0] PC_new;
wire[31:0] Rs1_data;
wire[31:0] ALU_B;

ImmGen_more ImmGen(
    .ImmSel(ImmSel),
    .inst_field(inst_field),
    .Imm_out(Imm_out)
);

add32 add_32_0(
    .a(32'h4),
    .b(PC_out),
    .c(PC_inc)
);

add32 add_32_1(
    .a(PC_out),
    .b(Imm_out),
    .c(PC_imm)
);

MUX2T1_32 MUX2T1_32_1(
    .I0(PC_inc),
    .I1(PC_imm),
    .s((Branch & ALU_zero) | (BranchN & ~ALU_zero)),
    .o(PC_1)
);

MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg),
    .I0(ALU_out),
    .I1(Data_in),
    .I2(PC_inc),
    .I3(Imm_out),
    .o(Wt_data)
);

```

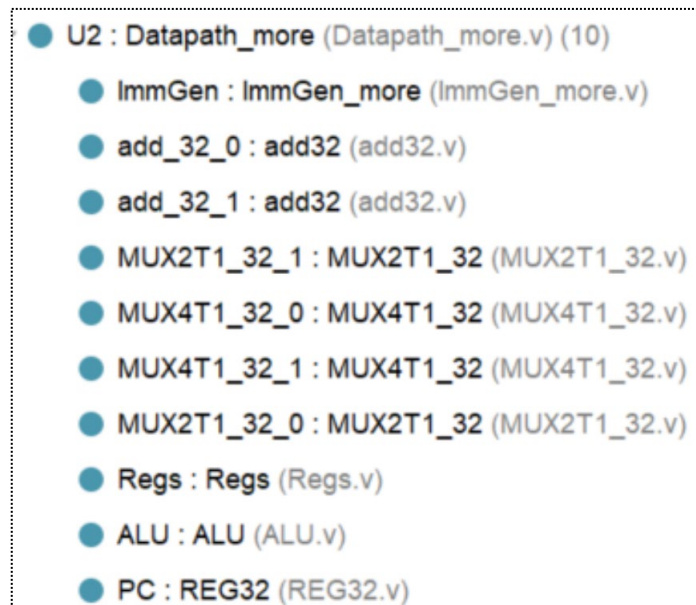
```

);
MUX4T1_32 MUX4T1_32_1(
    .I0(PC_1),
    .I1(PC_imm),
    .I2(ALU_out),
    .I3(PC_1),
    .s(Jump),
    .o(PC_new)
);
MUX2T1_32 MUX2T1_32_0(
    .I0(Data_out),
    .I1(Imm_out),
    .s(ALUSrc_B),
    .o(ALU_B)
);
Regs Regs(
    .clk(clk),
    .rst(rst),
    .RegWrite(RegWrite),
    .Rs1_addr(inst_field[19:15]),
    .Rs2_addr(inst_field[24:20]),
    .Wt_addr(inst_field[11:7]),
    .Wt_data(Wt_data),
    .Rs1_data(Rs1_data),
    .Rs2_data(Data_out),
    .all_regs_data(all_regs_data)
);
ALU ALU(
    .A(Rs1_data),
    .B(ALU_B),
    .ALU_operation(ALU_operation),
    .res(ALU_out),
    .zero(ALU_zero)
);
REG32 PC(
    .clk(clk),
    .rst(rst),
    .CE(1'b1),
    .D(PC_new),
    .Q(PC_out)
);

```

```
endmodule
```

(3) Lab04-3Datapath_more 工程结构:

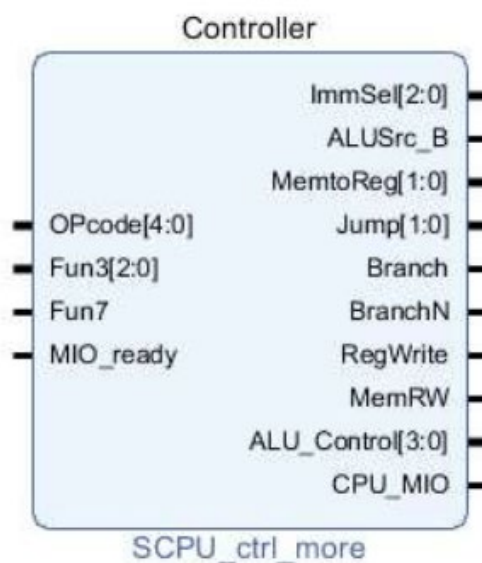


3.2 设计实现 SCPU_Ctrl:

(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-SCPU_ctrl_more;

本实验主要实现两个阶段的译码：根据指令 inst_in 对主要控制信号进行赋值分配；根据 ALU_op、Funct3/7 对 ALU 中的 ALU_Control 进行赋值。



(2) 描述设计 SCPU_ctrl_more:

- 逻辑接口:

```
module SCPU_ctrl_more(  
    input[4:0]OPcode, //Opcode  
    input[2:0]Fun3, //Function  
    input Fun7, //Function  
    input MIO_ready, //CPU Wait  
    output reg [2:0]ImmSel, //立即数选择控制  
    output reg ALUSrc_B, //源操作数 2 选择  
    output reg [1:0]MemtoReg, //写回数据选择控制  
    output reg [1:0]Jump, //jal && jalr  
    output reg Branch, //beq  
    output reg BranchN, // bne  
    output reg RegWrite, //寄存器写使能  
    output reg MemRW, //存储器读写使能  
    output reg [3:0]ALU_Control, //alu 控制  
    output reg CPU_MIO  
);  
reg [1:0] ALUop;  
wire [3:0] Fun;  
assign Fun = {Fun3, Fun7};
```

- 主控制器设计:

```
always @(*) begin  
    case(OPcode)  
5'b01100: begin  
        {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
        14'b0_00_1_0_0_0_00_10_000; end //ALU  
5'b00000: begin  
        {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
        14'b1_01_1_0_0_0_00_00_001; end //load  
5'b01000: begin  
        {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
        14'b1_00_0_1_0_0_00_00_010; end //store  
5'b11000: begin  
        case(Fun3)  
            3'b000: begin  
                {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
                14'b0_00_0_0_1_0_00_01_011; end // beq  
            3'b001: begin  
                {ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel} =  
                14'b0_00_0_0_0_1_00_01_011; end // bne  
            endcase  
        end  
    endcase
```

```

        end
5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_01_00_100; end //jump
5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_1_0_0_0_00_11_001; end //ALU(addi)
5'b11001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_10_00_001; end //jarl
5'b01101: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_11_1_0_0_0_00_00_000; end //lui
default:
begin {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} = 14'h0; end
        endcase
    end
end

```

- ALU 译码控制设计:

```

always @(*) begin
    case(ALUop)
        2'b00: ALU_Control = 4'b0010 ;
        2'b01: ALU_Control = 4'b0110 ;
        2'b10:
            case(Fun)
                4'b0000: ALU_Control = 4'b0010 ; //add
                4'b0001: ALU_Control = 4'b0110 ; //sub
                4'b1110: ALU_Control = 4'b0000 ; //and
                4'b1100: ALU_Control = 4'b0001 ; //or
                4'b0100: ALU_Control = 4'b0111 ; //slt
                4'b1010: ALU_Control = 4'b1101 ; //srl
                4'b1000: ALU_Control = 4'b1100 ; //xor
                4'b0110: ALU_Control = 4'b1001 ; //sltu
                4'b0010: ALU_Control = 4'b1110 ; //sll
                4'b1011: ALU_Control = 4'b1111 ; //sra
                default: ALU_Control = 4'bx;
            endcase
        2'b11:
    end
end

```

```

    case(Fun3)
      3'b000: ALU_Control = 4'b0010 ; //addi
      3'b010: ALU_Control = 4'b0111 ; //slti
      3'b100: ALU_Control = 4'b1100 ; //xori
      3'b110: ALU_Control = 4'b0001 ; //ori
      3'b111: ALU_Control = 4'b0000 ; //andi
      3'b101:
        if(Fun7)
          ALU_Control = 4'b1111 ; //srai
        else
          ALU_Control = 4'b1101 ; //srli
      3'b011: ALU_Control = 4'b1001 ; //sltiu
      3'b001: ALU_Control = 4'b1110 ; //slli
      default: ALU_Control = 4'bx ;
    endcase
  endcase
end

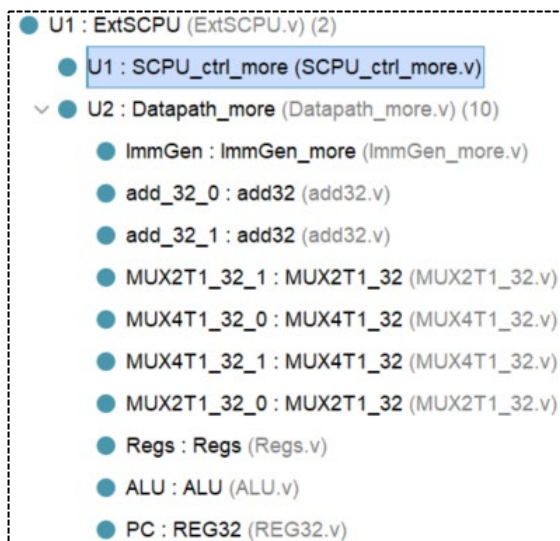
```

3.3 集成 CPU

3.3.1 搭建仿真平台

通过 3.1、3.2，我们通过逻辑代码实现了 Datapath_more 和 SCPU_ctrl_more 模块，对 CPU 进行了指令拓展，我们参考 Lab4-02 的操作也能建立一个 CPU 仿真测试平台，对相关指令进行检测。

首先对两个模块进行集成组合成 CPU：



【具体代码如下】

```
`timescale 1ns / 1ps

module ExtSCPU(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,

    output MemRW,
    output CPU_MIO,
    output [31:0]Data_out,
    output [31:0]PC_out,
    output [31:0]Addr_out,
    output [1023:0] all_regs_data
);

    wire [3:0]ALU_Control;
    wire [2:0]ImmSel;
    wire [1:0]MementoReg,Jump;
    wire Branch, BranchN, RegWrite, ALUSrc_B;

    SCPU_ctrl_more U1(
        .OPcode(inst_in[6:2]),
        .Fun3(inst_in[14:12]),
        .Fun7(inst_in[30]),
        .MIO_ready(MIO_ready),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MementoReg(MementoReg),
        .Jump(Jump),
        .Branch(Branch),
        .BranchN(BranchN),
        .RegWrite(RegWrite),
        .MemRW(MemRW),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO)
    );

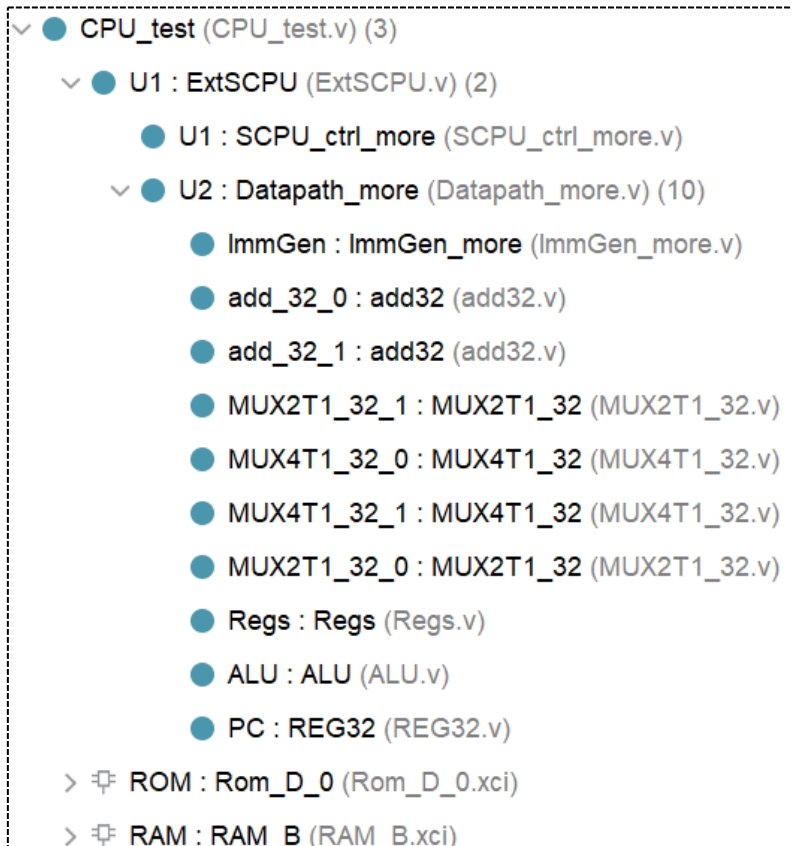
    Datapath_more U2(
        .ALUSrc_B(ALUSrc_B),
        .ALU_operation(ALU_Control),
        .Branch(Branch),
        .BranchN(BranchN),
```

```

        .Data_in(Data_in),
        .ImmSel(ImmSel),
        .Jump(Jump),
        .MemtoReg(MemtoReg),
        .RegWrite(RegWrite),
        .clk(clk),
        .inst_field(inst_in),
        .rst(rst),
        .ALU_out(Addr_out),
        .Data_out(Data_out),
        .PC_out(PC_out),
        .all_regs_data(all_regs_data)
    );
endmodule

```

同样的，利用 CPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台，初始化 RAM 的数据(D_mem.coe)，初始化 ROM 的指令(I_mem_more.coe)，查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。





【此处给出部分仿真结果，对于整套指令的分析见后一下板验证部分】

3.3.2 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。

本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格。

- ROM 中填充指令（I_mem_more.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
00007293, 00007313, 88888137, 00832183, 0032A223, 00402083, 01C02383, 00338863,
555550B7, 0070A0B3, FE0098E3, 007282B3, 00230333, 00531463, 40000033, 40530433,
405304B3, 0080006F, 00007033, 0072F533, 00157593, 00B51463, 00006033, 00A5E5B3,
0015E513, 00558463, 00004033, 00A5C633, 00164613, 00B61463, 00000013, 0012D293,
00060463, 40000033, 00129293, 00B28463, 00000013, 001026B3, 00503733, F65FF06F
```

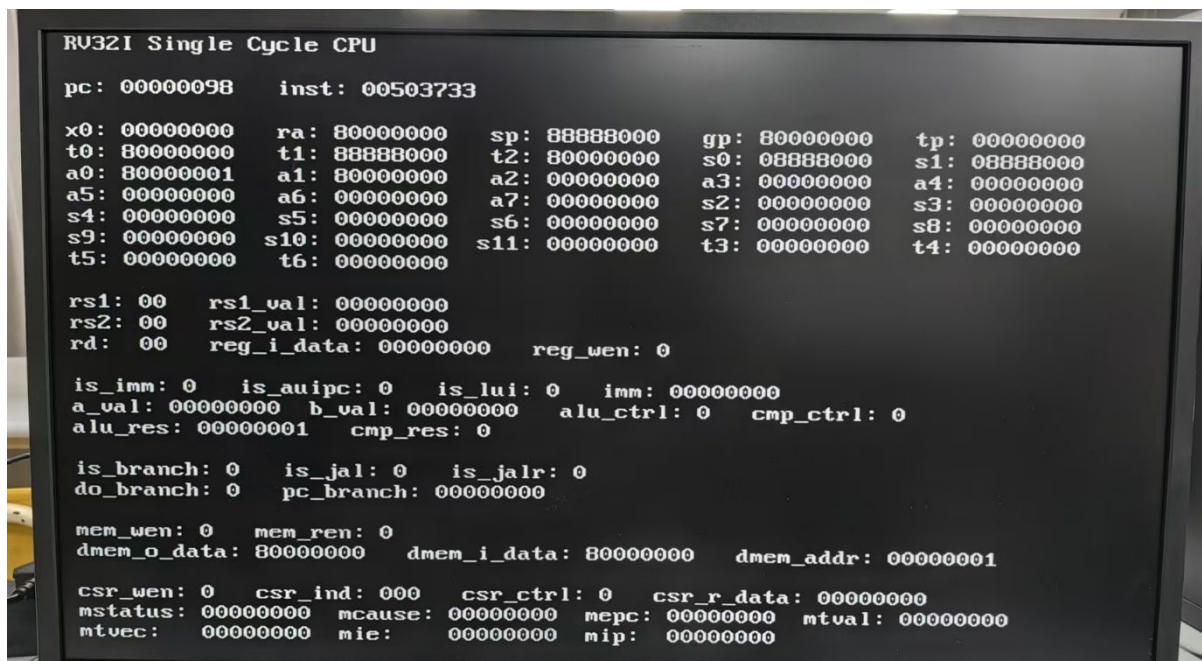
- RAM 中储存的数据（D_mem.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7DBFDB9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFBFCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7DBFDB9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300
```

部分测试 VGA 显示图片见实验结果分析。

四、实验结果分析

由于指令集较为冗长，此处放了最后 3 个指令的 VGA 显示图片，便于观察之前所有指令所修改的寄存器值是否正确。



【PC = 0x98 inst = 0x00503733 将要执行 sltu x14 , x0 , x5】

```

RV32I Single Cycle CPU
pc: 0000009c    inst: f65ff06f

x0: 00000000    ra: 80000000    sp: 88888000    gp: 80000000    tp: 00000000
t0: 80000000    t1: 88888000    t2: 80000000    s0: 08888000    s1: 08888000
a0: 80000001    a1: 80000000    a2: 00000000    a3: 00000000    a4: 00000001
a5: 00000000    a6: 00000000    a7: 00000000    s2: 00000000    s3: 00000000
s4: 00000000    s5: 00000000    s6: 00000000    s7: 00000000    s8: 00000000
s9: 00000000    s10: 00000000    s11: 00000000    t3: 00000000    t4: 00000000
t5: 00000000    t6: 00000000

rs1: 00    rs1_val: 00000000
rs2: 00    rs2_val: 00000000
rd: 00    reg_i_data: 00000000    reg_wen: 0

is_imm: 0    is_auiopc: 0    is_lui: 0    imm: 00000000
a_val: 00000000    b_val: 00000000    alu_ctrl: 0    cmp_ctrl: 0
alu_res: ffffff64    cmp_res: 0

is_branch: 0    is_jal: 0    is_jalr: 0
do_branch: 0    pc_branch: 00000000

mem_wen: 0    mem_ren: 0
dmem_o_data: 80000000    dmem_i_data: 00000000    dmem_addr: fffffff64

csr_wen: 0    csr_ind: 000    csr_ctrl: 0    csr_r_data: 00000000
mstatus: 00000000    mcause: 00000000    mepc: 00000000    mtval: 00000000
mtvec: 00000000    mie: 00000000    mip: 00000000

```

- (1) 执行完 PC = 0x98 后, x14 = a4 = 0x00000001,
- (2) PC = 0x9c, inst = 0XF65FF06F, 将要执行 jal x0 -156

```

RV32I Single Cycle CPU
pc: 00000000    inst: 00007293

x0: 00000000    ra: 80000000    sp: 88888000    gp: 80000000    tp: 00000000
t0: 80000000    t1: 88888000    t2: 80000000    s0: 08888000    s1: 08888000
a0: 80000001    a1: 80000000    a2: 00000000    a3: 00000000    a4: 00000001
a5: 00000000    a6: 00000000    a7: 00000000    s2: 00000000    s3: 00000000
s4: 00000000    s5: 00000000    s6: 00000000    s7: 00000000    s8: 00000000
s9: 00000000    s10: 00000000    s11: 00000000    t3: 00000000    t4: 00000000
t5: 00000000    t6: 00000000

rs1: 00    rs1_val: 00000000
rs2: 00    rs2_val: 00000000
rd: 00    reg_i_data: 00000000    reg_wen: 0

is_imm: 0    is_auiopc: 0    is_lui: 0    imm: 00000000
a_val: 00000000    b_val: 00000000    alu_ctrl: 0    cmp_ctrl: 0
alu_res: 00000000    cmp_res: 0

is_branch: 0    is_jal: 0    is_jalr: 0
do_branch: 0    pc_branch: 00000000

mem_wen: 0    mem_ren: 0
dmem_o_data: 80000000    dmem_i_data: 00000000    dmem_addr: 00000000

csr_wen: 0    csr_ind: 000    csr_ctrl: 0    csr_r_data: 00000000
mstatus: 00000000    mcause: 00000000    mepc: 00000000    mtval: 00000000
mtvec: 00000000    mie: 00000000    mip: 00000000

```

执行完整套指令, 跳转到 PC = 0 处, 同时寄存器组保存了所有运算值。

PC	Machine	Code Basic	Status	Result
0x0	0x00007293	andi x5 x0 0		x5 = 0
0x4	0x00007313	andi x6 x0 0		x6 = 0
0x8	0x88888137	lui x2 559240		x2 = 0x88888000
0xc	0x00832183	lw x3 8(x6)		x3 = 0x80000000
0x10	0x0032A223	sw x3 4(x5)		Mem[1] = 0x80000000
0x14	0x00402083	lw x1 4(x0)		x1 = 0x80000000
0x18	0x01C02383	lw x7 28(x0)		x7 = 0x80000000
0x1c	0x00338863	beq x7 x3 16		go to 2c
0x20	0x555550B7	lui x1 349525	Jump	
0x24	0x0070A0B3	slt x1 x1 x7	Jump	
0x28	0xFE0098E3	bne x1 x0 -16	Jump	
0x2c	0x007282B3	add x5 x5 x7		x5 = 0x80000000
0x30	0x00230333	add x6 x6 x2		x6 = 0x88888000
0x34	0x00531463	bne x6 x5 8		go to 3c
0x38	0x40000033	sub x0 x0 x0	Jump	
0x3c	0x40530433	sub x8 x6 x5		x8 = 0x08888000
0x40	0x405304B3	sub x9 x6 x5		x9 = 0x08888000
0x44	0x0080006F	jal x0 8		x0 no change
0x48	0x00007033	and x0 x0 x0	Jump	
0x4c	0x0072F533	and x10 x5 x7		x10 = 0x80000000
0x50	0x00157593	andi x11 x10 1		x11 = 0x0
0x54	0x00B51463	bne x10 x11 8		go to 5c
0x58	0x00006033	or x0 x0 x0	Jump	
0x5c	0x00A5E5B3	or x11 x11 x10		x11 = 0x80000000
0x60	0x0015E513	ori x10 x11 1		x10 = 0x80000001
0x64	0x00558463	beq x11 x5 8		go to 6c
0x68	0x00004033	xor x0 x0 x0	Jump	
0x6c	0x00A5C633	xor x12 x11 x10		x12 = 0x00000001
0x70	0x00164613	xori x12 x12 1		x12 = 0x0
0x74	0x00B61463	bne x12 x11 8		go to 7c
0x78	0x00000013	addi x0 x0 0	Jump	
0x7c	0x0012D293	srli x5 x5 1		x5 = 0x40000000
0x80	0x00060463	beq x12 x0 8		go to 88
0x84	0x40000033	sub x0 x0 x0	Jump	
0x88	0x00129293	slli x5 x5 1		x5 = 0x80000000
0x8c	0x00B28463	beq x5 x11 8		go to 94
0x90	0x00000013	addi x0 x0 0	Jump	
0x94	0x001026B3	slt x13 x0 x1		x13 = 0x0 (signed x1 is minus)
0x98	0x00503733	sltu x14 x0 x5		x14 = 0x00000001
0x9c	0xF65FF06F	jal x0 -156		go to 0

上表记录了整套指令的具体操作，相关结果可以对照 Result 和 VGA 显示中寄存器组的相关结果。

该实验成功运行了基础指令和拓展的指令（lui、bne、sltu、srli 等），DEMO 测试程序能够顺利执行，实验结果符合预期。

五、讨论与心得

思考题 1：指令扩展时控制器用二级译码设计存在什么问题？

随着指令拓展，二级译码的中间量的位宽需要不断增加，同时如果要拓展同一指令类型中的指令时，可能存在译码级数增加的可能，如 `srai` 和 `srli` 需要经过 `ALUop---Fun3---Fun7` 的多层连续译码。

```
2'b11:
    case(Fun3)
        3'b000: ALU_Control = 4'b0010 ; //addi
        3'b010: ALU_Control = 4'b0111 ; //slti
        3'b100: ALU_Control = 4'b1100 ; //xori
        3'b110: ALU_Control = 4'b0001 ; //ori
        3'b111: ALU_Control = 4'b0000 ; //andi
        3'b101:
            if(Fun7)
                ALU_Control = 4'b1111 ; //srai
            else
                ALU_Control = 4'b1101 ; //srli
```

思考题 2：设计 `bne` 指令需要增加控制信号吗？

需要添加一个类似 Branch 的控制信号 `BranchN`。

`Beq` 判定跳转的条件是 `zero & Branch = 1`（利用一个 `and`），如果不添加控制信号，`Branch = 1`、`zero = 0` 无法实现 `bne` 跳转。

因此添加 `BranchN`，在判定 PC 跳转的 MUX 处，添加一个 `or`，使得 `BranchN = 1`、`zero = 0` 时也能够利用 `BranchN & (! zero)` 来判定 `bne` 的跳转。

思考题 2：设计 `srai` 时需要增加新的数据通道吗？

`Srai` 作为 I-Type 中的一项指令，需要 `ImmGen` 来生成立即数，并递交给 ALU 进行右移操作，因此不需要添加新的数据通路。

指令拓展实验看似不难，只需要修改数据通路，并更新控制器中的控制信号。但是战线比较长，导致会有小错误出现难以纠正，因此仿真还是比较重要的。

最终下板能看到正确结果（一样的代码两次跑 bit 下板结果不一样？），还是比较惊喜的，革命尚未成功，同志还需努力！